



HASHING BASICS & TECHNICAL REFERENCE

CRYPTOGRAPHIC DIGEST MECHANICS & CRACKING OPERATIONS

1. CORE PRINCIPLES OF CRYPTOGRAPHIC HASHING

- **Deterministic:** A given input always produces the exact same hash output [cite: x].
- **One-Way Function:** Computationally infeasible to reverse $Hash(M) \rightarrow M$ [cite: x].
- **Pre-image Resistance:** Given a hash output h , it is computationally difficult to find any input M such that $H(M) = h$ [cite: x].
- **Collision Resistance:** Computationally infeasible to find two distinct inputs $M_1 \neq M_2$ such that $H(M_1) = H(M_2)$ [cite: x].
- **Avalanche Effect:** Flipping a single bit in the input radically transforms the resulting hash digest output [cite: x].

2. PASSWORD STORAGE PROTECTION MECHANICS

METHOD	VULNERABILITY / ARCHITECTURE	SECURITY ASSESSMENT
Plaintext	Direct exposure upon database leak [cite: x].	NON-COMPLIANT / INSECURE [cite: x]
Symmetric Encryption	Requires secret key storage; compromised key exposes all data [cite: x].	INSECURE [cite: x]
Unsalted Hashes	Vulnerable to precomputed lookups (Rainbow Tables) [cite: x].	INSECURE [cite: x]
Salted Hashes	Uniquely appends a random string per user before hashing [cite: x].	INDUSTRY STANDARD [cite: x]

Salting Mechanics: $Calculated\ Digest = Hash\ Function(Plaintext\ Password + Salt)$ [cite: x]

- **Rainbow Table Defeat:** Invalidates precomputed global hash tables by requiring distinct tables computed specifically per individual salt [cite: x].
- **Salt Storage:** Stored unencrypted alongside the hash digest within user record databases [cite: x].

3. DATA INTEGRITY & MESSAGE AUTHENTICATION CODES (HMAC)

File Integrity Checking: Hashes act as unique cryptographic checksums [cite: x]. If a downloaded file matches the published hash, integrity is verified [cite: x].

HMAC (Keyed-Hash Message Authentication Code): Combines a secret cryptographic key (K) with a hash algorithm (H) to guarantee both Integrity (data unaltered) and Authenticity (data originated from keyholder) [cite: x].

$$HMAC(K, M) = H((K \oplus opad) \parallel H((K \oplus ipad) \parallel M)) \text{ [cite: x]}$$

K : Secret key (padded to algorithm block size) [cite: x]

M : Target message [cite: x]

$\oplus$: Bitwise XOR operator [cite: x]

$\parallel$: String concatenation operator [cite: x]

$ipad / opad$: Inner and outer fixed padding constants [cite: x]

4. HASH FORMAT IDENTIFICATION

LINUX SHADOW STRUCTURE (/ETC/SHADOW) [cite: x]

$SprefixOptionsSalt$hash$ [cite: x]

Prefix Identifiers: [cite: x]

- $\$y\$$ = yescrypt (Modern default) [cite: x]
- $\$gy\$$ = gost-yescrypt [cite: x] | $\$7\$$ = scrypt [cite: x]
- $\$2b\$ / \$2y\$ / \$2a\$$ = bcrypt [cite: x]
- $\$6\$$ = sha512crypt (SHA-512) [cite: x]
- $\$1\$$ = md5crypt (Deprecated) [cite: x]

WINDOWS PASSWORDS (SAM DATABASE) [cite: x]

NTLM: 32-character hexadecimal digest derived from MD4 [cite: x].

Extracted via runtime memory injection (**lsass.exe**) or offline SAM registry hive analysis [cite: x].

> 5. COMMAND-LINE REFERENCE & TOOL USAGE

INSPECTION, DIAGNOSTICS & BASE64

```
# Hexdump binary viewer | Paginated text viewer
hexdump -C filename [cite: x]
less -N filename [cite: x]

# Word, line, character, byte counter (-w, -m, -l, -c)
sha256sum filename | wc -wm1c [cite: x]

# Standard Integrity Checksums
md5sum filename [cite: x] | shasum filename [cite: x] | sha256sum filename [cite: x]

# Base64 Encode & Decode
echo -n "data" | base64 [cite: x]
echo "ZGF0YQ==" | base64 -d [cite: x]
```

HASH IDENTIFICATION TOOLS

```
# hashID (Python length/signature lookup)
hashid -m "5b31f93c09ad1d065c0491b764d04933" [cite: x]

# nth (Name That Hash - provides Hashcat/John flags directly)
nth -t "5b31f93c09ad1d065c0491b764d04933" [cite: x]
```

JOHN THE RIPPER

```
# Wordlist Attack
john --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt [cite: x]

# Explicit Format & Show
john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt [cite: x]
john --show --format=raw-md5 hash.txt [cite: x]

# Unshadow Utility (/etc/passwd + /etc/shadow)
unshadow /etc/passwd /etc/shadow > unshadowed_hashes.txt [cite: x]
john --wordlist=/usr/share/wordlists/rockyou.txt unshadowed_hashes.txt [cite: x]
```

HASHCAT REFERENCE

```
# Syntax Structure
hashcat -m <hash_type> -a <attack_mode> <hashfile> <wordlist> [cite: x]

# Modes (-a): 0=Wordlist, 1=Combo, 3=Brute-force/Mask [cite: x]
# Selected Types (-m): 0=MD5, 100=SHA1, 1400=SHA256, 10000=NTLM, 1800=sha512crypt, 3200=bcrypt [cite: x]

# MD5 & NTLM Wordlist Attacks
hashcat -m 0 -a 0 hash.txt /usr/share/wordlists/rockyou.txt [cite: x]
hashcat -m 1000 -a 0 ntlm_hashes.txt /usr/share/wordlists/rockyou.txt [cite: x]

# Display Cracked Hashes from Cache
hashcat -m 0 --show hash.txt [cite: x]
```